

BOTFLOW

Documento de Conocimiento del Proyecto

Este documento contiene la arquitectura completa, funcionalidades, esquema de base de datos, flujos de datos y detalles de implementación del proyecto **Botflow** — una plataforma SaaS multi-tenant para crear y gestionar bots de WhatsApp con IA. Generado el **1 de junio de 2026**.

Uso: Cargar este PDF en un chat nuevo para continuar el desarrollo con contexto completo.

1. Resumen Ejecutivo

Botflow es una plataforma SaaS construida con **Next.js 14 (App Router)** que permite a negocios crear agentes de WhatsApp impulsados por Claude (Anthropic). Cada usuario puede tener múltiples bots, cada uno con su propia personalidad, base de conocimientos, catálogo, flujo de conversación y sistema de citas.

Stack tecnológico principal

Framework	Next.js 14 (App Router, JavaScript)
Auth	Clerk (@clerk/nextjs ^6.39.5)
Base de datos	Supabase (PostgreSQL) — migración desde Vercel KV
IA / LLM	Anthropic Claude (claude-haiku-4-5-20251001) vía API REST directa
Voz	OpenAI Whisper (transcripción de notas de voz WhatsApp)
Pagos	Stripe (checkout, subscriptions, webhooks)
Deploy	Vercel (vercel --prod)
Mensajería	WhatsApp Business API (Meta Graph API v19.0)
Google	Calendar API + Sheets API (service account JWT, sin SDK npm)
Parsing docs	pdf-parse, mammoth, xlsx, cheerio

2. Arquitectura del Sistema

2.1 Estructura de archivos (App Router)

app/	
page.js	← Landing pública
layout.js	← Root layout (Clerk Provider)
sign-in / sign-up	← Auth pages (Clerk)
onboarding/page.js	← Onboarding inicial
booking/page.js	← Página pública de reserva de citas
conectar/page.js	← WhatsApp Embedded Signup
dashboard/	
layout.js	← Layout del dashboard (sidebar)
page.js	← Dashboard principal
bots/page.js	← Lista de bots del usuario
bots/[id]/page.js	← Detalle / configuración de un bot
create/page.js	← Crear nuevo bot
settings/page.js	← Configuración global
knowledge/page.js	← Base de conocimientos
analytics/page.js	← Analíticas de mensajes
citas/page.js	← Gestión de citas
planes/page.js	← Planes y suscripciones
admin/page.js	← Admin panel
api/	
webhook/route.js	← Webhook WhatsApp (GET verify + POST messages)
bots/route.js	← CRUD lista de bots (GET, PATCH, DELETE)
bots/[id]/route.js	← Detalle bot (GET, PUT)
config/route.js	← Config global
create-bot/route.js	← Onboarding bot (step-by-step + autofill)
create-bot/autofill/	← Autofill con scraping
chat/route.js	← Chat de prueba interno
knowledge/route.js	← CRUD base de conocimientos
knowledge/image/[id]/	← Servir imágenes KB
scrape/route.js	← Scraping de sitios web
settings/autofill/	← Autofill settings desde URL
analytics/route.js	← Analytics data
appointments/route.js	← Booking de citas
booking-config/route.js	← Config pública de citas
booking-slots/route.js	← Slots disponibles (pública)
onboarding/route.js	← Onboarding
stripe/checkout/route.js	← Stripe Checkout Session
stripe/portal/route.js	← Stripe Customer Portal
stripe/receipt/route.js	← Stripe receipt
stripe/webhook/route.js	← Stripe webhooks
subscription/route.js	← Estado suscripción
whatsapp-signup/route.js	← WhatsApp Embedded Signup
whatsapp-profile/route.js	← Perfil WA
favicon/route.js	← Servir favicon del bot
landing/route.js	← Config landing page
lib/	
storage.js	← Capa de datos (Supabase + KV fallback)
supabase.js	← Cliente Supabase (Service Role)
crypto.js	← AES-256-GCM encrypt/decrypt
webhookAuth.js	← Meta signature + HMAC ownership
google.js	← Google Calendar + Sheets (JWT directo)
bookAppointment.js	← Lógica booking de citas
scripts/	

```
migrate-kv-to-supabase.js ← Migración KV → Supabase  
backfill-ownership-hmac.js ← Backfill HMACs en phone_mappings
```

2.2 Middleware (Clerk Auth)

El archivo **middleware.js** usa Clerk para proteger todas las rutas del dashboard. Las rutas públicas son:

- / (landing)
- /sign-in, /sign-up
- /booking (reserva pública)
- /onboarding, /conectar
- /api/webhook (WhatsApp)
- /api/stripe/webhook
- /api/booking-config, /api/booking-slots
- /api/appointments, /api/favicon, /api/chat

3. Base de Datos — Supabase Schema v2

El sistema está en proceso de migración de **Vercel KV** a **Supabase (PostgreSQL)**. Se activa con la env var `USE_SUPABASE=true`. El flag controla todas las operaciones en `lib/storage.js`.

users

Mirror de usuarios Clerk. `id` = Clerk `userId` (string). Campos: `id`, `email`, `plan` (free/starter/pro/enterprise), `stripe_customer_id`, `created_at`, `updated_at`.

bots

Un usuario puede tener múltiples bots. Soft delete con `deleted_at`. Campos: `id`, `user_id`, `name`, `agent_name`, `business_name`, `status` (ACTIVO/INACTIVO), `phone_number_id`, `wa_business_id`, `display_phone`, `message_count`, `conversation_count`, `deleted_at`, `created_at`, `updated_at`.

bot_configs

Configuración de personalidad y negocio por bot. `UNIQUE(bot_id)`. Campos: `agent_name`, `tone`, `language`, `greeting`, `extra_instructions`, `use_emojis`, `short_answers`, `faqs` (JSONB), `business_name/desc/profile`, `services`, `hours`, `location`, `full_address`, `phone`, `website`, `contact_email`, `price_policy`, `instagram`, `facebook`, `tiktok`, `youtube`, `catalog` (JSONB), `flow` (JSONB), `appointments` (JSONB), `site_name`, `favicon_base64`, `favicon_mime_type`.

bot_secrets

Claves sensibles cifradas con AES-256-GCM. `UNIQUE(bot_id)`. Campos: `access_token_enc`, `anthropic_key_enc`, `openai_key_enc`, `verify_token`.

phone_mappings

Mapeo `phone_number_id` → `user_id` + `bot_id`. PK: `phone_number_id`. Incluye `hmac_sig` para verificación de propiedad anti-hijack.

knowledge_files

Índice de archivos/imágenes subidos por usuario. Campos: `id`, `user_id`, `bot_id`, `name`, `type`, `size`, `status` (PROCESANDO/PROCESADO/ERROR), `text_file`, `preview`, `is_image`, `mime_type`.

knowledge_content

Contenido textual extraído de archivos. `UNIQUE(user_id, file_key)`. Imágenes se almacenan como base64 JSON con key `img:{id}`.

conversations

Historial de conversación por usuario/bot/teléfono. `UNIQUE(user_id, bot_id, from_phone)`. TTL: `expires_at` (7 días). Limpiado por `pg_cron`.

analytics

Contadores atómicos vía RPC `increment_analytics`. Campos: `total_messages`, `total_conversations`, `daily_counts` (JSONB).

message_events

Log individual de mensajes para queries de `recent_messages`. Evita race condition en JSONB.

subscriptions

Suscripciones Stripe por usuario. Campos: plan, status, stripe_customer_id, stripe_subscription_id, current_period_end.

audit_log

Auditoría de cambios en bot_configs y bot_secrets.

3.1 RLS (Row Level Security)

Todas las tablas tienen RLS habilitado con política deny-all para roles anon/authenticated. Solo el **Service Role Key** (usado server-side en lib/supabase.js) tiene acceso. NUNCA exponer el Service Role al browser.

3.2 Cifrado de secretos (lib/crypto.js)

Formato del ciphertext: **v1:<base64(iv[12] + ciphertext + authTag[16])>**

Algoritmo: AES-256-GCM. La clave (ENCRYPTION_KEY) es un hex de 64 chars almacenado SOLO en Vercel env vars. Soporte de rotación de claves via KEY_VERSIONS map.

4. Flujo del Webhook WhatsApp

Archivo: `app/api/webhook/route.js`

4.1 Verificación (GET)

Meta envía GET con `hub.mode=subscribe`, `hub.verify_token`, `hub.challenge`. Se verifica contra `WA_VERIFY_TOKEN` de env vars. Responde con el challenge.

4.2 Recepción de mensajes (POST) — pasos en orden

1. Verificar Meta Signature	<code>verifyMetaSignature(rawBody, X-Hub-Signature-256)</code> . Usa <code>META_APP_SECRET</code> + HMAC-SHA256 con <code>timingSafeEqual</code> . Si falla: 403.
2. Extraer mensaje	Soporta: text, audio (notas de voz), interactive (<code>button_reply</code> / <code>list_reply</code>).
3. Resolver userId por phoneId	<code>getUserIdByPhone(phoneId)</code> desde <code>phone_mappings</code> . Fallback: env <code>WA_PHONE_NUMBER_ID</code> → <code>userId='default'</code> .
4. Verificar ownership HMAC	<code>verifyOwnershipHmac(phoneId, userId, storedHmac)</code> . Previene hijacking de <code>phone_number_id</code> entre usuarios. HMAC = HMAC-SHA256(<code>phoneId:userId, WEBHOOK_SECRET</code>).
5. Resolver bot activo	<code>getBotIdByPhone(phoneId)</code> → busca en bots del usuario. Fallback: primer bot ACTIVO.
6. Transcripción audio	Si <code>tipo=audio</code> : descarga media de Meta API → envía a OpenAI Whisper <code>/v1/audio/transcriptions</code> (<code>whisper-1</code> , <code>language=es</code>).
7. Cargar contexto	<code>getAllKBText(userId)</code> + <code>getKBImages(userId)</code> + <code>getConversation(from, userId)</code> + <code>getAvailableSlots()</code> si citas habilitadas.
8. Llamar Claude	<code>callClaude(text, config, knowledge, kbImages, history, availableSlots)</code> → <code>claude-haiku-4-5-20251001</code> , <code>max_tokens=600</code> .
9. Parsear respuesta de Claude	Detecta markers: <code>[BOOK_APPOINTMENT:{...}]</code> , <code>[SEND_BOOKING_LINK]</code> , <code>[SEND_IMAGE:id]</code> .
10. Enviar respuesta	<code>sendWhatsAppText</code> + (opcional) <code>sendWhatsAppMenu</code> + booking link + imágenes.
11. Guardar historial	<code>setConversation(from, updatedHistory, userId)</code> . <code>trackMessage()</code> para analytics.

4.3 System Prompt de Claude

El prompt incluye en este orden:

- Identidad: nombre del agente, negocio, descripción, perfil, servicios, horario, ubicación, teléfono, web, política de precios.
- Catálogo de productos/servicios (con `imageId` si aplica).
- Tono, idioma, emojis, respuestas cortas.

- Saludo personalizado (solo primer mensaje). Siempre fuerza el agentName correcto.
- REGLA CRÍTICA: NUNCA cierres la conversación ante un rechazo — siempre continúa con estrategia.
- REGLA CRÍTICA: Mostrar servicios con entusiasmo y enviar imágenes automáticamente.
- FAQs predefinidas.
- Instrucciones especiales (extraInstructions).
- Modo de flujo: text / menu / hybrid.
- Sistema de citas con campos, horarios disponibles y formato [BOOK_APPOINTMENT:{...}].
- Base de conocimientos (texto completo, hasta 40,000 chars).
- Lista de imágenes disponibles con IDs exactos y reglas de envío.

5. Gestión de Bots (Multi-Bot)

Cada usuario puede tener **múltiples bots**. Cada bot tiene su propia configuración independiente en `bot_configs`, sus secretos en `bot_secrets`, y su mapeo de teléfono en `phone_mappings`.

5.1 API de Bots

GET /api/bots	Devuelve lista de bots del usuario autenticado (filtrado por <code>user_id</code>).
PATCH /api/bots	Cambia status (ACTIVO/INACTIVO) de un bot.
DELETE /api/bots	Elimina un bot de la lista.
GET /api/bots/[id]	Detalle del bot. Fusiona config + bot row. Si falta <code>displayPhone</code> , lo consulta de Meta API.
PUT /api/bots/[id]	Actualiza bot. Auto-corrige greeting si cambia <code>agentName</code> . Sincroniza campos a config global. Actualiza <code>phone_mappings</code> si cambió <code>phoneNumberId</code> .

5.2 Bug conocido: bots migrados

Durante la migración de KV a Supabase, los bots del ID 'default' (usuario sin Clerk) quedaron con `user_id` genérico. Resultado: múltiples usuarios pueden ver el mismo bot migrado si la query no filtra correctamente por `user_id`. **Fix pendiente:** asegurar que `getBots()` siempre filtra `.eq('user_id', uid)` y que los bots migrados tengan el `user_id` correcto.

Datos migrados con problema:

```
[
  { id: 'bot-migrated-ydQcs8PI-...', name: 'Karya design', user_id: 'user_3EKe...' },
  { id: 'bot-migrated-default-...', name: 'pyl', user_id: 'user_3EKe...', agent_name: 'Karielys' },
  { id: 'bot-migrated-eYC7ljCs-...', name: 'pyl', user_id: 'user_3ETw...' }, ← DIFERENTE USER
  { id: 'bot-migrated-ZXIqsrC-...', name: 'CarveMaker', user_id: 'user_3EYI...' }
]
```

El bot 'pyl' aparece para dos usuarios distintos — indica que ambos tienen un bot con el mismo `business_name` pero diferente `user_id`. Es correcto en datos pero el UI debe filtrar correctamente. Revisar si el `GET /api/bots` en KV fallback devuelve todos los bots sin filtro de usuario.

5.3 Creación de bots

Hay dos modos:

- **Step-by-step (legacy):** `POST /api/create-bot` con `step/userAnswer`. Claude Haiku responde a cada pregunta y al final construye config.
- **Autofill (nuevo):** `POST /api/create-bot` con `_autofill:true` + `answers` completo. Guarda directamente sin llamar a Claude. También hace scraping de URL via `/api/create-bot/autofill`.

6. Base de Conocimientos

La KB es **por usuario** (no por bot todavía — `bot_id` es nullable en `knowledge_files`). Soporta: PDF, DOCX, XLSX, sitios web (scraping), imágenes.

GET /api/knowledge	Lista archivos del usuario (<code>getKBIndex</code>).
POST /api/knowledge	Sube archivo. Extrae texto según tipo: <code>pdf-parse</code> , <code>mammoth</code> , <code>xlsx</code> , <code>scraping cheerio</code> . Guarda en <code>knowledge_content</code> .
DELETE /api/knowledge	Elimina archivo del índice y su contenido.
GET /api/knowledge/image/[id]	Devuelve imagen del KB como base64 para el bot (<code>getKBImageData</code>).
POST /api/scrape	Scraping de URL con <code>cheerio</code> . Extrae texto limpio.

Las imágenes se almacenan como JSON en `knowledge_content` con key `img:{id}`:

```
{ "base64": "...", "mimeType": "image/jpeg", "filename": "foto.jpg" }
```

El webhook puede enviar imágenes a WhatsApp via **sendWhatsAppImage** que sube la imagen primero a WhatsApp Media API y luego la envía como mensaje tipo image.

7. Sistema de Citas

Configurado en `bot_configs.appointments` (JSONB). El bot puede agendar citas directamente en WhatsApp o via link de reserva pública (`/booking`).

appointments.enabled	boolean — activa el sistema
appointments.fields	Array de campos a recolectar: {key, label, required}
appointments.calendarId	ID del Google Calendar donde crear eventos
appointments.googleCredentials	JSON del service account de Google (guardado cifrado)
appointments.sheetId	Google Sheets ID para registro de citas (opcional)

7.1 Flujo de booking vía WhatsApp

- Claude recolecta los campos requeridos uno por uno.
- Claude puede ofrecer horarios disponibles via `getAvailableSlots()` (Google Calendar free/busy).
- Al tener todos los datos, Claude incluye `[BOOK_APPOINTMENT:{...}]` al final de su respuesta.
- El webhook parsea el marker y llama `bookAppointment()` directamente (sin HTTP roundtrip).
- `bookAppointment()` crea evento en Google Calendar, registra en Google Sheets, y envía confirmación por WhatsApp.
- Alternativamente: `[SEND_BOOKING_LINK]` envía URL de la página `/booking?from={phone}`.

7.2 Página pública /booking

Permite elegir fecha/hora visualmente. Consulta `/api/booking-slots` (pública) para slots disponibles. POST `/api/appointments` para confirmar. No requiere autenticación.

8. Flujos de Conversación

Configurado en bot_configs.flow (JSONB). Tres modos:

text	Modo por defecto. Conversación libre con Claude.
menu	Siempre muestra menú interactivo de WhatsApp (buttons o list). Claude responde a la opción seleccionada.
hybrid	Muestra menú solo en el primer mensaje. Después conversación libre.

flow config schema

```
{
  "mode": "text" | "menu" | "hybrid",
  "menuType": "buttons" | "list",
  "menuWelcome": "Selecciona una opción:",
  "menuItems": [
    { "id": "1", "title": "Ver servicios", "description": "...", "response": "..." }
  ]
}
```

Límites WhatsApp: buttons → máx 3 items, título máx 20 chars. list → máx 10 items, título máx 24 chars, descripción máx 72 chars.

9. Suscripciones y Planes (Stripe)

Planes	free, starter, pro, enterprise
Checkout	POST /api/stripe/checkout — crea Checkout Session. priceId resuelto server-side desde env vars (STRIPE_PRICE_STARTER, _PRO, _ENTERPRISE).
Portal	POST /api/stripe/portal — Customer Portal de Stripe para gestionar suscripción.
Webhook	POST /api/stripe/webhook — maneja checkout.session.completed, customer.subscription.updated/deleted. Actualiza tabla subscriptions.
Estado	GET /api/subscription — devuelve plan/status del usuario autenticado.

10. Variables de Entorno Requeridas

SUPABASE_URL	URL del proyecto Supabase
SUPABASE_SERVICE_KEY	Service Role Key de Supabase (bypasses RLS)
USE_SUPABASE	'true' para activar Supabase. 'false' usa KV fallback
ENCRYPTION_KEY	Hex 64 chars para AES-256-GCM (secretos de bots)
WEBHOOK_SECRET	String 32+ chars para HMAC ownership de phone mappings
META_APP_SECRET	App Secret de Meta Developer Console (firma webhook)
WA_VERIFY_TOKEN	Token de verificación del webhook WhatsApp
WA_ACCESS_TOKEN	Access Token global de WhatsApp (fallback)
WA_PHONE_NUMBER_ID	Phone Number ID global de WhatsApp (fallback)
WA_BUSINESS_ID	Business ID de WhatsApp (fallback)
ANTHROPIC_KEY	API Key de Anthropic (fallback global)
OPENAI_API_KEY	API Key de OpenAI para Whisper (fallback)
STRIPE_SECRET_KEY	Secret key de Stripe
STRIPE_WEBHOOK_SECRET	Signing secret del webhook de Stripe
STRIPE_PRICE_STARTER	Price ID plan Starter en Stripe
STRIPE_PRICE_PRO	Price ID plan Pro en Stripe
STRIPE_PRICE_ENTERPRISE	Price ID plan Enterprise en Stripe
KV_REST_API_URL	URL de Vercel KV (solo mientras USE_SUPABASE=false)
KV_REST_API_TOKEN	Token de Vercel KV (solo mientras USE_SUPABASE=false)
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	Clerk publishable key
CLERK_SECRET_KEY	Clerk secret key
VERCEL_PROJECT_PRODUCTION_URL	URL de producción (para links de booking)

11. Seguridad

Meta Signature	Cada POST del webhook verifica X-Hub-Signature-256 con HMAC-SHA256(rawBody, META_APP_SECRET). timingSafeEqual previene timing attacks.
Ownership HMAC	phone_mappings.hmac_sig = HMAC-SHA256(phoneId:userId, WEBHOOK_SECRET). Verificado en cada mensaje entrante para prevenir que un usuario registre el teléfono de otro.

Cifrado en BD	access_token, anthropic_key, openai_key siempre cifrados con AES-256-GCM. La ENCRYPTION_KEY nunca toca Supabase.
RLS Supabase	Deny-all para anon/authenticated. Solo service_role accede. El servidor nunca expone el Service Role Key al cliente.
Clerk JWT	Todas las rutas del dashboard protegidas por Clerk middleware. userId extraído server-side via auth().
Stripe Webhook	Verificado con stripe.webhooks.constructEvent(rawBody, sig, STRIPE_WEBHOOK_SECRET).

12. Analytics

Los contadores de mensajes se actualizan vía RPC atómica **increment_analytics** en Supabase para evitar race conditions. Los eventos individuales se guardan en **message_events** para queries de mensajes recientes.

trackMessage()	Llamado por el webhook tras cada respuesta. Detecta si es conversación nueva ese día.
getAnalytics()	Devuelve totalMessages, totalConversations, dailyCounts, recentMessages(50).
GET /api/analytics	Endpoint del dashboard. Filtra por userId (+ botId opcional).

13. Migración KV → Supabase

Script: **scripts/migrate-kv-to-supabase.js**

Migra en este orden: configs, bot lists, phone mappings, knowledge base, conversations, subscriptions. Soporta `--dry-run` y `--verbose`.

El script cifra `access_token` y `anthropic_key` con `encrypt()` antes de insertar en `bot_secrets`.

Pasos para completar migración

- 1. Aplicar `supabase/schema.sql` en Supabase SQL Editor.
- 2. Correr: `node scripts/migrate-kv-to-supabase.js --dry-run`
- 3. Si OK: `node scripts/migrate-kv-to-supabase.js`
- 4. Correr: `node scripts/backfill-ownership-hmac.js`
- 5. Setear `USE_SUPABASE=true` en Vercel env vars.
- 6. Deploy y monitorear logs 30 minutos.

Estado actual: La migración ya corrió y se obtuvieron 4 bots migrados (ver sección 5.2). Pendiente verificar que `USE_SUPABASE=true` esté activo y que el filtro por `user_id` funcione correctamente en el UI.

14. WhatsApp Embedded Signup

Página: **/conectar** — Flujo de Facebook Embedded Signup para que el usuario conecte su WhatsApp Business sin salir de la app. API: POST `/api/whatsapp-signup` (intercambia código por access token, registra phone mapping). API: POST `/api/whatsapp-profile` (actualiza foto/nombre del perfil WhatsApp del bot).

15. Integración Google (lib/google.js)

NO usa npm packages de Google. Implementa auth JWT de service account directamente via fetch a `googleapis.com`. Scopes usados: Calendar API + Sheets API.

getGoogleToken(creds, scopes)	Genera JWT, intercambia por <code>access_token</code> en <code>oauth2.googleapis.com/token</code> .
getAvailableSlots(appointments)	Consulta free/busy del calendar para los próximos 5 días. Retorna string con horarios disponibles para el system prompt.
createCalendarEvent(data, token)	Crea evento en Google Calendar con título, descripción, fecha/hora, duración.
appendToSheet(data, token)	Agrega fila a Google Sheets con los datos de la cita.
invalidateSlotsCache()	Invalida cache de slots después de booking.

16. Configuración Local y Desarrollo

npm run dev	Next.js dev server en localhost:3000
npm run build	Build de producción
vercel --prod	Deploy directo a Vercel
botflow-config.json	Archivo de config local (solo dev, fuera de git). Contiene config de ejemplo de Producciones YL.
botflow-bots.json	Lista de bots local (vacía en el repo).

En local dev sin KV_REST_API_URL, storage.js usa archivos JSON locales (botflow-config_{uid}.json, botflow-bots_{uid}.json, etc.)

add-env.js

Script utilitario para agregar variables de entorno a Vercel vía CLI. Permite setear múltiples env vars en bulk para los environments preview/production.

17. Dependencias Clave (package.json)

@clerk/nextjs ^6.39.5	Auth — usuario, sesión, middleware
@vercel/kv ^3.0.0	KV store legacy (fallback durante migración)
@supabase/supabase-js	Cliente Supabase (instalado pero no en pkg.json — verificar)
stripe ^17.7.0	Pagos y suscripciones
pdf-parse ^1.1.4	Extracción de texto de PDFs
mammoth ^1.12.0	Extracción de texto de DOCX
xlsx ^0.18.5	Lectura de archivos Excel
cheerio ^1.2.0	Scraping de sitios web
next 14.2.5	Framework — App Router
react ^18	UI
tailwindcss ^3.4.1	Estilos (utility-first CSS)

Nota: @supabase/supabase-js no aparece en package.json pero se usa en lib/supabase.js. Verificar que esté instalado: `npm install @supabase/supabase-js`